

## 9. Well-Behaved Objects

### Example 9.1

```
public class Calculator {  
  
    public static int divide(int a, int b) {  
        return a / b;  
    }  
  
    public static void main(String[] args) {  
        int result = divide(10, 2);  
        System.out.println(result);  
    }  
}
```

This example covers

- A program to be **tested**
  - Possible **syntax error** (grammar mistake)
  - Possible **logical error** (wrong logic)
  - Method calls and **call sequence**
  - Can be tested manually or by **unit testing**
  - Can be tested automatically using **JUnit**
- 

### Example 9.2

```
import static org.junit.Assert.*;  
import org.junit.Test;  
  
public class CalculatorTest {  
  
    @Test  
    public void testDivide() {  
        assertEquals(5, Calculator.divide(10, 2));  
    }  
}
```

This example covers

- **Unit testing**
  - **JUnit**
  - **Test automation**
  - Positive test case
- 

## 9.1. Program Errors

## Definition

A program error is a mistake that causes incorrect behavior or failure.

---

### 9.1.1. # Syntax Error

#### Definition

A syntax error breaks the grammar rules of Java.

#### Features

- Detected by the compiler
- Program cannot run

#### Example

```
int x = 10
```

- Missing semicolon
- 

### 9.1.2. # Logical Error

#### Definition

A logical error means the program runs but produces a wrong result.

#### Features

- No compiler error
- Harder to find

#### Example

```
int area = length + width;
```

- Formula is wrong
- 

#### Remark

- Syntax errors stop compilation
  - Logical errors require testing to find
- 

## 9.2. Testing

## Definition

Testing is the process of running a program to find errors.

### Features

- Compares actual output with expected output
  - Does not fix errors
- 

### 9.2.1. # Positive Testing

#### Definition

Positive testing uses valid input to check correct behavior.

#### Example

```
divide(10, 2);
```

- Expected result: 5
- 

### 9.2.2. # Negative Testing

#### Definition

Negative testing uses invalid or extreme input.

#### Example

```
divide(10, 0);
```

- Tests error handling
- 

#### Remark

- Good testing includes both positive and negative tests
- 

## 9.3. Debugging

### Definition

Debugging is the process of finding and fixing errors.

### Features

- Performed after testing

- Requires understanding code logic
- 

### 9.3.1. # Manual Walkthrough

#### Definition

A manual walkthrough means reading code line by line and simulating execution.

#### Features

- No tools required
  - Useful for small programs
- 

### 9.3.2. # Call Sequence

#### Definition

The call sequence is the order in which methods are executed.

#### Example

```
main → divide → return result
```

#### Use

- Helps locate where an error occurs
- 

#### Remark

- Debugging fixes errors
  - Testing only finds errors
- 

## 9.4. Unit Testing

### Definition

Unit testing tests one small unit of code, usually one method.

### Features Components of a Unit Test

- A well-defined unit test is composed of three sequential components:
- Arrange: preparation of the context and the specific data inputs necessary for testing the unit. It includes defining concrete inputs to test the unit.
- Act: this is the execution of the unit being tested. It typically just involves calling the software under test in the prepared context with the selected data.

- Assert:
    1. captures the expectations on the execution of the test, i.e., the expected behavior (if the software were correct).
    2. It requires understanding precisely what the software is supposed to do in the given context for the given data
    3. Simple asserts check expected output against actual output
- 

### 9.4.1. # JUnit

#### Definition

JUnit is a Java framework for unit testing.

#### Features

- Uses `@Test`
- Uses assertions like `assertEquals`

#### Example

```
@Test
public void testAdd() {
    assertEquals(4, add(2, 2));
}
```

---

#### Remark

- Each unit test should test only one behavior
- 

## 9.5. Test Automation

#### Definition

Test automation means running tests automatically using tools.

#### Features

- Fast execution
  - Repeatable tests
  - No manual input
- 

### 9.5.1. # Regression Testing

#### Definition

Regression testing re-runs old tests after code changes.

## ***Purpose***

- Ensure old features still work
  - Prevent new bugs
- 

## **Remark**

- Automated tests are ideal for regression testing
- 

## **Chapter 9 — Core Exam Summary**

- Syntax error vs logical error
- Testing vs debugging
- Meaning of unit testing
- Role of JUnit
- Positive vs negative testing
- Purpose of regression testing
- Meaning of manual walkthrough
- Meaning of call sequence